Notifire.in

Real-time Tech Intelligence Platform

Technical Documentation

Workflow Overview & Technology Stack

Notifire.in is an **"Inshorts for Tech"** platform that aggregates real-time technology news from 15+ RSS sources across 7 categories, uses AI to summarize and analyze articles, detects trending stories, and generates unique illustrations — all within a 24-hour rolling window.

> **Core Principle:** The platform never copies or embeds source images directly. Instead, it generates unique AI-created illustrations inspired by article content, avoiding copyright issues while providing visually rich presentation.

15+	7	8	24h
RSS Sources	Categories	Pipeline Steps	Rolling Window

1. How It Works — The Complete Workflow

The system follows an **8-step pipeline** split into two phases. **Steps 1–5** run automatically every 5 minutes to fetch, process, and store articles. **Steps 6–8** are triggered on-demand when a user clicks an article, providing deep AI enrichment only when needed.

AUTOMATIC (Every 5 min)	ON-DEMAND (User Click)
Steps 1–5: Fetch → Process → Store	Steps 6–8: Scrape → Summarize → Generate

1 RSS Feed Aggregation

The pipeline starts by fetching articles from 15+ RSS and Atom feed sources organized across 7 categories. Each source has a pre-configured authority score (0–1) reflecting its credibility.

- All feeds are fetched in parallel using Promise.allSettled for speed.
- Each feed is parsed to extract title, URL, description, publication date, author, and images.
- A 24-hour cutoff filter discards articles older than 24 hours immediately.
- There is no per-feed limit — every qualifying article within the window is included.
- Failed feeds are logged but do not block the pipeline.

2 Category Detection & Tagging

Each article is assigned to one of 7 categories (AI, Cybersecurity, Cloud, Databases, Infrastructure, DevOps, Startup) using a keyword-based detection algorithm, then tagged with the top 5 most relevant keywords.

- Each category has 15–25 weighted keywords (e.g., AI: "llm", "gpt", "openai", "chatgpt", "deepseek").
- The system counts keyword matches in the article title + description.
- The category with the highest match count wins; fallback is "AI".
- Top 5 matching keywords become article tags displayed as hashtags.

3 Deduplication

The same story is often covered by multiple sources. A two-layer deduplication strategy combines exact URL matching with Jaccard similarity on titles.

- Exact URL match → immediate duplicate detection.
- Title tokenization → Jaccard similarity = |intersection| / |union|.
- If similarity > 0.7 (70% overlap), the article is considered a duplicate.
- Non-duplicates get a deterministic ID from SHA-256 hash of their URL.

4 Trending Score Calculation

Each surviving article receives a trending score (0–100) from four independent components. Scores above 50 are marked "Rising" and above 65 as "Hot".

- Recency (0–30): Newer articles score higher. max(0, 30 − hoursAgo × 0.5).
- Source Authority (0–20): Pre-configured authority × 20. MIT Tech Review = 18.4 pts.
- Cross-Source Frequency (0–30): min(30, matchingArticles × 10). Multiple sources covering same topic = boost.
- Velocity Keywords (0–20): min(20, matches × 10). Keywords like "breaking", "launch", "breach", "zero-day".

5 24-Hour Filter & Database Persistence

A hard 24-hour rolling window is applied, and all qualifying articles are upserted to the PostgreSQL database.

- Articles published more than 24 hours ago are removed (both server and client-side).
- Each article is upserted using its unique URL as the key — updates title, description, score, tags.
- New records include all metadata; AI fields (summary, content, image) remain empty for on-demand filling.
- The frontend auto-refreshes every 5 minutes with incremental updates via a "since" timestamp.

6 Article Scraping (On-Demand)

When a user clicks an article, the system checks if full content is already in the database. If not, it scrapes the article from its original URL using the z-ai page reader.

- The z-ai-web-dev-sdk page_reader function retrieves full HTML content from the URL.
- Raw HTML is processed: scripts/styles/tags removed, entities decoded → clean plain text.
- Extracted text is capped at 8,000 characters for AI token limits.
- Word count and reading time (~200 wpm) are calculated and saved.

7 AI Summarization & Analysis (On-Demand)

Once full content is available, the AI generates a comprehensive analysis including summary, key points, sentiment, and actionable drafts for LinkedIn and email.

- Full summary: 3–4 sentences covering what happened, why it matters, and impact.
- 5 key points: Bullet points under 15 words each.
- Sentiment: positive / neutral / negative.
- 4 related topics for discovery and navigation.
- LinkedIn post draft with 3–5 hashtags (under 200 words).
- Email newsletter draft with subject line and 3–4 sentence body.
- All output is cached in the database — repeat visits return instantly.

8 AI Image Generation (On-Demand / Auto)

The platform never copies source images. Instead, unique AI illustrations are generated inspired by article content. For the top 3 trending articles, images are generated automatically; for others, on user request.

- Image prompt = article subject + category-specific visual style + editorial illustration style.
- Each category has a distinct style (AI: futuristic neural networks/purple-cyan; Cybersecurity: shields/firewalls/red-dark).
- Generated at 1344×768 pixels via z-ai-web-dev-sdk image generation.
- Saved to /public/generated/ with MD5 hash filenames — never regenerated from disk cache.

2. Technology Stack

Frontend

Technology	Version	Purpose
Next.js	16.x	App Router, Server Components, API Routes, ISR
React	19.x	UI library with concurrent rendering
TypeScript	5.x	Strict type safety across entire codebase
Tailwind CSS	4.x	Utility-first CSS with theme variables
shadcn/ui	Latest	Radix-based component library (50+ components)
Framer Motion	12.x	Animations, transitions, AnimatePresence
Lucide React	Latest	Icon library (30+ icons)
next-themes	Latest	Dark/light mode switching

Backend

Technology	Version	Purpose
Next.js API Routes	16.x	7 REST endpoints (GET/POST/PUT/DELETE)
Prisma ORM	6.x	Type-safe DB queries with driver adapters
PGlite (PostgreSQL WASM)	0.4.x	In-process PostgreSQL with filesystem persistence
pglite-prisma-adapter	0.7.x	Bridges PGlite ↔ Prisma Client
z-ai-web-dev-sdk	0.0.17	AI: LLM chat, page reader, image generation
rss-parser	3.x	RSS/Atom feed fetching and parsing
sharp	0.34.x	Server-side image processing

AI Capabilities

Capability	SDK Function	Usage
LLM Chat Completions	chat.completions.create()	Summarization, key points, sentiment, drafts
Page Reader	functions.invoke("page_reader")	Full article content extraction from URLs
Image Generation	images.generations.create()	AI-generated article illustrations (1344×768)

3. Database & Data Model

Notifire.in uses **PGlite** — PostgreSQL compiled to WebAssembly — running in-process within the Node.js application. This eliminates the need for a separate database server while providing full PostgreSQL

compatibility.

Advantage	Description
No separate server	PGlite runs in-process as part of the Next.js app
Full PostgreSQL	Indexes, constraints, GROUP BY, aggregations all work
Filesystem persistence	Data stored at .pgdata/ survives restarts
Graceful fallback	Falls back to in-memory mode if filesystem fails

Article Model — Key Fields

Field	Type	Populated By	Description
title, url, description, source	String	Step 1 (RSS)	Basic article metadata
category, tags	String	Step 2 (Tagger)	Category label + keyword tags
trendScore	Float	Step 4 (Trending)	0–100 trending score
content	Text	Step 6 (Scraping)	Full article text (8k char cap)
summary, keyPoints, sentiment	String/JSON	Step 7 (AI)	AI-generated analysis
aiImageUrl	String	Step 8 (Image Gen)	Path to AI-generated illustration

4. API Endpoints

Method	Endpoint	Purpose
GET	/api/news	Fetch all 24h articles with category filter & incremental updates
GET	/api/trending	Trending articles + source/category distribution signals
POST	/api/detail	On-demand: scrape → AI summarize → return full detail
POST	/api/scrape	Scrape single URL using AI page reader
PUT	/api/scrape	Batch scrape up to 10 URLs with rate limiting
POST	/api/summarize	AI summarize article (optional scrape-first mode)
POST/PUT	/api/generate-image	Generate AI illustration for articles
GET/POST /DELETE	/api/saved	Bookmark management for articles

5. Caching & Performance

Layer	TTL	Stores	Invalidation
In-Memory Cache	5min / 1hr	API response data with expiry timestamps	Time-based + ?refresh=true

ISR (Next.js)	300s	Server-rendered page cache	Auto background revalidation
Database Cache	Permanent	AI summaries, scraped content, image paths	Upsert on re-scrape
Image Disk Cache	Permanent	AI-generated PNG files in /public/generated/	Never (MD5 filenames)
Client Incremental	5min cycle	New articles merged via ?since= parameter	Full refresh on tab change

6. News Sources by Category

Category	Sources	Authority Range
AI	TechCrunch AI, VentureBeat AI, Ars Technica, MIT Tech Review	0.85–0.92
Cybersecurity	The Hacker News, Krebs on Security, Dark Reading, Schneier	0.85–0.92
Cloud	AWS Blog, The New Stack	0.80–0.90
Databases	PostgreSQL.org, The New Stack, Redis Blog	0.75–0.85
Infrastructure	Kubernetes Blog, DevOps.com, The New Stack, InfoQ	0.78–0.90
DevOps	DevOps.com, InfoQ, HashiCorp Blog	0.80–0.88
Startup	TechCrunch Startups, Hacker News (YC)	0.85–0.88

7. Frontend Experience

Tab	Data Source	Shows
24h Feed	/api/news	All articles from last 24 hours, auto-refresh every 5 min
AI Live	/api/news?withImages=true	Same feed with AI enrichment indicators
Trending	/api/trending	Articles by trend score + velocity signals
Saved	/api/saved	Bookmarked articles, no 24h filter
Docs	—	Interactive documentation + PDF download

User Flow: Browse feed → filter by category/search → click article → on-demand enrichment triggers (scrape + AI summarize + image gen) → view full analysis → save bookmark or copy LinkedIn/email drafts → auto-refresh brings new articles every 5 minutes.

8. Pipeline Summary

Step	Name	Trigger	Output
1	RSS Feed Aggregation	Every 5 min	Raw articles from 15+ sources (24h window)
2	Category Detection	Every 5 min	Category label + 5 keyword tags per article

3	Deduplication	Every 5 min	Unique articles (Jaccard 0.7 threshold)
4	Trending Scoring	Every 5 min	Trend score (0–100) per article
5	24h Filter + DB Persist	Every 5 min	Database records with all metadata
6	Article Scraping	On user click	Full article text + reading time
7	AI Summarization	On user click	Summary, key points, sentiment, drafts
8	AI Image Generation	On click / auto (top 3)	1344×768 AI illustration saved to disk

End of Documentation — Notifire.in v1.0